

pandas



*not this one

INTRO

integrated structural framework for statistics, data
manipulation and analysis



name

The name is derived from the term **panel data**.

An *econometrics* term for data sets that include observations over multiple time periods for the same individuals

why pandas

STATISTICIAN are not **PROGRAMMER**

PROGRAMMER are not **STATISTIAN**

history

Out of the need In 2008 **Wes McKinney** started working on high performance, flexible tool to perform quantitative analysis on financial data while working at **AQR Capital Management**

packages

```
python -m venv venv  
venv/bin/pip install pandas
```

alternative installation using conda, active state or
system packages

starting

```
import pandas as pd # convention  
print(pd.__version__)
```

2.3.0

DATAFRAME



intro

there is columns and index like in spreadsheet

create

```
import pandas as pd

color = pd.Series({
    'R' : 'red', 'Y' : 'yellow', 'G' : 'green',
})
df = pd.DataFrame({
    'color' : color,
    'signal' : {'R' : 'stop', 'Y' : 'wait', 'G' : 'go'},
    'person' : {'R' : 'go', 'Y' : 'stop', 'G' : 'stop'}
})
print(df)
```

	color	signal	person
R	red	stop	go
Y	yellow	wait	stop
G	green	go	stop

structure

```
print(df.index)
print(df.columns)
print(df.columns.values)
```

```
Index(['R', 'Y', 'G'], dtype='object')
Index(['color', 'signal', 'person'], dtype='object')
['color' 'signal' 'person']
```

column

```
print(df['signal'])  
print(type(df.signal)) # lookout for name collision  
print(df.signal.to_dict())
```

```
R    stop  
Y    wait  
G    go  
Name: signal, dtype: object  
<class 'pandas.core.series.Series'>  
{'R': 'stop', 'Y': 'wait', 'G': 'go'}
```

ndarray

```
print(df.values)  
print(type(df.values))
```

```
[['red' 'stop' 'go']  
 ['yellow' 'wait' 'stop']  
 ['green' 'go' 'stop']]  
<class 'numpy.ndarray'>
```

transpose

```
print(df.T)
```

	R	Y	G
color	red	yellow	green
signal	stop	wait	go
person	go	stop	stop

1. create dataframe as show

	foo	bar
a	0	1
b	2	3
c	4	5

DATA SOURCE



sqlite

```
import sqlite

import pandas as pd

connection = psycopg2.connect('postgresql://postgres@localhost/po
df = pd.read_sql('SELECT 1 AS hello, 2 as world', connection)
connection.close()
print(df)
```

postgres

```
import pandas as pd

import psycopg2

connection = psycopg2.connect('postgresql://postgres@localhost/po
df = pd.read_sql('SELECT 1 AS hello, 2 as world', connection)
connection.close()
print(df)
```

	hello	world
0	1	2

spreadsheet

```
import pandas as pd

# requires : pip install openpyxl
df = pd.read_excel(
    io      = './data/books.xlsx',
    sheet_name = 'novel',
    index_col = 0,
)
print(df.info())
```

<class 'pandas.core.frame.DataFrame'>

Int64Index: 50 entries, 1 to 50

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	title	50 non-null	object
1	author	50 non-null	object
2	year	50 non-null	int64
3	guttenberg	50 non-null	object
4	language	50 non-null	object
5	title-en	20 non-null	object

dtypes: int64(1), object(5)

csv load

```
import pandas as pd

df = pd.read_csv(
    './data/companies.tsv',
    delimiter = '\t',
)
print(df)
```

	quote		name	sector
0	ABBL	Annapurna Bikash Bank	Limited	development bank
1	ACE	Ace Finance Company	Limited	finance
2	ACEDBL	ACE Development Bank	Limited	development bank
3	ADBL	Agriculture Development Bank	Limited	commercial bank
4	AEFL	Alpic Everest Finance Company	Limited	finance
..	...		...	...
435	YETI	Yeti Development Bank	Limited	development bank
436	YFL	Yeti Finance Company	Limited	finance
437	YHL	Yak and Yeti Hotel	Limited	hotel
438	ZFL	Zenith Finance	Limited	finance
439	SFCL	Samriddhi Finance Company	Limited	finance

[440 rows x 5 columns]

DF: WRANGLING COL



create

```
print(df)
```

	A	B	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

read

```
print(df['A'])
```

```
1    A1  
2    A2  
3    A3  
Name: A, dtype: object
```

slice

similar to numpy and series

```
print(df[['C', 'B']])
```

	C	B
1	C1	B1
2	C2	B2
3	C3	B3

adding columns

```
df['list'] = [ 4, 5, 6 ]
df['range'] = range(3)
df['scalar'] = 'new'
df['split'] = list('new')
print(df)
```

	A	B	C	D	list	range	scalar	split
1	A1	B1	C1	D1	4	0	new	n
2	A2	B2	C2	D2	5	1	new	e
3	A3	B3	C3	D3	6	2	new	w

rename

```
df.rename(  
    columns = {  
        'A' : 'Ant',  
        'B' : 'Bat',  
    },  
    inplace = True,  
)  
print(df)
```

	Ant	Bat	C	D
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3

del columns

```
del df['C']  
df.drop(columns=['D'], inplace=True)  
print(df)
```

	A	B
1	A1	B1
2	A2	B2
3	A3	B3

examples

```
import datetime as dt
import pandas as pd

df = pd.read_csv(
    './data/floorsheet.tsv',
    encoding = 'utf-8',
    delimiter = '\t',
    index_col = ['ts'],
)

df['day'] = df.index.map(
    lambda r: dt.datetime.strptime(r[:10], '%Y-%m-%d')
)
print(df['day'])
```

```
ts
2024-11-25T10:46:00.002865    2024-11-25
2024-11-25T10:46:00.012527    2024-11-25
2024-11-25T10:46:00.015159    2024-11-25
2024-11-25T10:46:00.019797    2024-11-25
2024-11-25T10:46:00.022646    2024-11-25
...
2024-11-25T14:59:59.498902    2024-11-25
```

DF: WRANGLING ROW



load data

```
import pandas as pd

df = pd.read_csv(
    './data/companies.tsv',
    delimiter = '\t',
    index_col = 'quote',
)

print(df)
```

		name	sector
quote			
ABBL	Annapurna Bikash Bank Limited	development bank	
ACE	Ace Finance Company Limited	finance	
ACEDBL	ACE Development Bank Limited	development bank	
ADBL	Agriculture Development Bank Limited	commercial bank	
AEFL	Alpic Everest Finance Company Limited	finance	
...	...	...	...
YETI	Yeti Development Bank Limited	development bank	
YFL	Yeti Finance Company Limited	finance	
YHL	Yak and Yeti Hotel Limited	hotel	
ZFL	Zenith Finance Limited	finance	
SFCL	Samriddhi Finance Company Limited	finance	

rows

```
print(df.shape)
print(df.head(2))
print(df.tail(2))
```

```
(440, 2)
```

	name	sector
quote		
ABBL	Annapurna Bikash Bank Limited	development bank
ACE	Ace Finance Company Limited	finance
	name	sector
quote		
ZFL	Zenith Finance Limited	finance
SFCL	Samriddhi Finance Company Limited	finance

by-index

```
print(df.name['ADBL'])  
print(df.T['ADBL']['name'])  
print(df.loc['ADBL'])  
print(df.loc['ADBL', 'name'])  
print(df.iloc[1])
```

```
Agriculture Development Bank Limited  
Agriculture Development Bank Limited  
name      Agriculture Development Bank Limited  
sector                                commercial bank  
Name: ADBL, dtype: object  
Agriculture Development Bank Limited  
name      Ace Finance Company Limited  
sector                                finance  
Name: ACE, dtype: object
```

Using conditions to filter rows.

DF: GROUPBY



aggregate method

```
import pandas as pd

df = pd.DataFrame(
    data = {
        'key' : list('ABC'*3),
        'val' : [1] * 9,
    },
)
print(df.sum())
print(df.groupby('key').sum())
```

```
key    ABCABCABC
val          9
dtype: object
1.0
    val
key
A      3
B      3
C      3
```

count companies in each sector

sector	count
Finance	1
commercial bank	37
debenture	1
development bank	122
finance	97
hotel	4
hydropower	44
insurance	3
life insurance	8
manufacturing and processing	31
micro finance	45

load data

```
import pandas as pd

df = pd.read_csv(
    './data/floorsheet.tsv',
    delimiter = '\t',
    index_col = 1,
)
df.index = pd.to_datetime(df.index//100000000, format='%Y%m%d')
df.index.name = 'day'
print(df.head())
```

	ts	nid	symbol	...	volume	rate
day				...		
2024-11-25	2024-11-25T10:46:00.002865	131	NABIL	...	10	569.1
2024-11-25	2024-11-25T10:46:00.012527	134	HBL	...	10	257.0
2024-11-25	2024-11-25T10:46:00.015159	135	SBI	...	10	468.1
2024-11-25	2024-11-25T10:46:00.019797	139	NICA	...	10	435.5
2024-11-25	2024-11-25T10:46:00.022646	140	MBL	...	10	264.9

[5 rows x 8 columns]

describe

```
df.describe()  
# df.describe().unstack()
```

	buyer	seller	volume	rate	am
count	79967.000000	79967.000000	79967.000000	79967.000000	7.996700
mean	33.997211	35.118386	181.444008	921.197276	1.131458
std	17.563136	16.428157	681.518601	1143.868107	2.940966
min	1.000000	1.000000	1.000000	9.070000	2.230000
25%	18.000000	22.000000	20.000000	330.000000	1.205500
50%	37.000000	38.000000	55.000000	562.000000	4.560000
75%	49.000000	49.000000	188.000000	1225.000000	1.170000
max	59.000000	59.000000	75000.000000	19074.000000	4.497411

agg methods :resize this thing:

a reduce function many to one

method	description
count	Total number of items
first, last	First and last item
mean, median	Mean and median
min, max	Minimum and maximum
std, var	Standard deviation and variance
mad	Mean absolute deviation

method	description
prod	Product of all items
sum	Sum of all items

groupby

```
print(df.groupby('symbol'))
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x775dbe146e40>
```


group-object

```
for (name, group) in df.groupby('symbol'):
    print(name, type(group))
    print(group.head())
    break
```

ACLBSL <class 'pandas.core.frame.DataFrame'>

	cid	nid	...	rate	amount
ts			...		
2024-11-25T11:12:59.606849	2024112503000521	2790	...	1113.0	14469.
2024-11-25T11:47:27.223639	2024112503002608	2790	...	1120.0	14560.
2024-11-25T12:07:11.190224	2024112503003698	2790	...	1120.0	11200.
2024-11-25T12:07:26.723567	2024112503003714	2790	...	1120.0	14560.
2024-11-25T12:17:09.141299	2024112503004180	2790	...	1115.1	55755.

[5 rows x 8 columns]

ADBL

groupby again

```
print(df.groupby('symbol')['rate'].describe())
```

	count	mean	std	...	50%	75%	max
symbol				...			
ACLBSL	35.0	1119.805714	4.117462	...	1120.0	1121.00	1130.0
ADBL	219.0	351.381279	1.604428	...	351.6	352.00	357.0
ADBLD83	4.0	1137.500000	2.886751	...	1137.5	1140.00	1140.0
AHL	94.0	696.988298	3.552883	...	697.9	700.00	702.0
AHPC	340.0	279.023235	1.794343	...	279.4	280.00	283.0
...	...	...	...	...	...	...	...
USHL	185.0	819.323784	6.536925	...	818.1	825.00	834.0
USLB	83.0	1621.938554	10.994509	...	1618.0	1630.05	1643.2
VLBS	67.0	945.908955	5.857131	...	948.0	950.00	953.0
VLUCL	251.0	653.174900	4.343654	...	652.0	656.00	664.0
WNLB	49.0	2412.244898	12.959119	...	2415.5	2422.00	2448.0

[309 rows x 8 columns]

aggregate

```
import numpy as np

print(
    df.groupby('symbol')[['volume', 'rate']].agg(
        ['min', np.median, np.max]
    )
)
```

	volume			rate		
symbol	min	median	max	min	median	max
ACLBSL	10	14.0	158	1113.0	1120.0	1130.0
ADBL	1	80.0	1387	343.7	351.6	357.0
ADBLD83	50	100.0	150	1135.0	1137.5	1140.0
AHL	10	30.0	410	687.2	697.9	702.0
AHPC	7	122.0	19861	273.5	279.4	283.0
...	...	...	...	...	...	...
USHL	1	40.0	850	794.0	818.1	834.0
USLB	10	20.0	1378	1603.0	1618.0	1643.2
VLBS	10	37.0	744	935.1	948.0	953.0
VLUCL	10	10.0	1571	643.0	652.0	664.0
WNLB	1	10.0	160	2386.0	2415.5	2448.0

aggregate with dict

```
import numpy as np

df.groupby('symbol').agg({
    'volume' : np.sum,
    'rate'    : np.mean,
    # 'rate'   : [ min, max ],
})
```

	volume	rate
quote		
ADBL	104745	500.474419
AHPC	79939	211.938776
AIL	25396	657.945896
AKJCL	9575	123.460606
AKPL	155878	251.289308
...	...	...
UNL	140	18781.142857
UPCL	93519	126.707101
UPPER	262863	394.233211
USLB	6343	1296.698795
VLBS	16043	1200.476744

[198 rows x 2 columns]

filter

```
print('-')
print(df[df.volume<10].head(3))
# print((df.groupby('quote')['volume'].sum() > 10).head())
print(df.groupby('symbol').filter(lambda r: r.volume.sum() == 100
```

```
-
```

	quote	buyer	seller	volume	rate	amount
cid						
2020122003000020	CIT	35	19	6	5700.0	34200.0
2020122003000124	CIT	58	42	5	5700.0	28500.0
2020122003001880	MSLB	22	16	4	1582.0	6328.0
	quote	buyer	seller	volume	rate	amount
cid						
2020122004000001	SFMF	10	16	100	10.60	1060.0
2020122004001328	NMB50	16	10	100	10.09	1009.0

each columns

```
print(df[['symbol', 'rate']].groupby('symbol').agg(  
    open = ('rate', lambda x: x.iloc[0]),  
    high = ('rate', 'max'),  
    low = ('rate', 'min'),  
    close = ('rate', lambda x: x.iloc[-1])  
))
```

	open	high	low	close
symbol				
ACLSL	1113.0	1130.0	1113.0	1116.0
ADBL	357.0	357.0	343.7	351.9
ADBLD83	1140.0	1140.0	1135.0	1135.0
AHL	689.0	702.0	687.2	700.0
AHPC	282.0	283.0	273.5	281.0
...	...	...	...	...
USHL	794.0	834.0	794.0	810.0
USLB	1642.2	1643.2	1603.0	1639.0
VLBS	950.0	953.0	935.1	951.6
VLUCL	643.0	664.0	643.0	646.0
WNLB	2425.0	2448.0	2386.0	2400.0

EXAMPLE

load data

```
import pandas as pd

df = pd.read_excel(
    io      = './data/books.xlsx',
    sheet_name = 'novel',
    index_col = 0,
)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
Index: 50 entries, 1 to 50
```

```
Data columns (total 6 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	title	50 non-null	object
1	author	50 non-null	object
2	year	50 non-null	int64
3	gutenberg	50 non-null	object
4	language	50 non-null	object
5	title-en	20 non-null	object

```
dtypes: int64(1), object(5)
```

```
memory usage: 2.7+ KB
```


which books as mostly written in

```
g = df.groupby('language')  
print(*g.groups.items(), sep='\n')
```

```
('Chinese', Int64Index([1, 5], dtype='int64', name='# '))  
('English', Int64Index([ 3,  4,  6,  7,  8,  9, 10, 12, 13, 19, 20, 21,  
                        39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49],  
                        dtype='int64', name='# '))  
('French', Int64Index([11, 14, 15, 16, 17, 18, 23, 25, 29, 30, 38], dtype='int64', name='# '))  
('Italian', Int64Index([35], dtype='int64', name='# '))  
('Japanese', Int64Index([50], dtype='int64', name='# '))  
('Russian', Int64Index([24, 26, 28, 31, 33, 34], dtype='int64', name='# '))  
('Spanish', Int64Index([2], dtype='int64', name='# '))
```

show count

title					
	count	author count	year count	guttenberg count	title-en count
language					
Chinese	2	2	2	2	2
English	28	28	28	28	0
French	11	11	11	11	10
Italian	1	1	1	1	1
Japanese	1	1	1	1	0
Russian	6	6	6	6	6
Spanish	1	1	1	1	1

MISSING DATA



why

real-world data is rarely clean and homogeneous

find nan

method	description
isna()	boolean mask with n/a
notna()	boolean mask with is available
dropna()	return not null values

null = na

```
import pandas as pd

print(pd.isnull) # alias of isna
print(pd.notnull) # alias of notna
```

```
<function isna at 0x7fa7a9823af0>
<function notna at 0x7fa7a9832d30>
```

series

```
import pandas as pd

s = pd.Series([1, float('inf'), float('nan'), None, 'text'])
print(s[s.isna()]) # masking
print(s.dropna())
```

```
2      NaN
3      None
dtype: object
0      1
1      inf
4      text
dtype: object
```

dataframe

```
import numpy as np
import pandas as pd

df = pd.DataFrame([
    [1, np.nan, 2],
    [2,      3, 5],
    [np.nan, 4, 6]
])
print(df.dropna())
print(df.dropna(axis='columns'))
```

	0	1	2
1	2.0	3.0	5
2			
0	2		
1	5		
2	6		

drop how

```
import numpy as np
import pandas as pd

df = pd.DataFrame([
    [1, np.nan, 2, np.nan],
    [2, 3, 5, np.nan],
    [np.nan, 4, 6, np.nan]
])
print(df.dropna(axis='columns', how='all'))
print(df.dropna(axis='rows', thresh=3))
```

	0	1	2
0	1.0	NaN	2
1	2.0	3.0	5
2	NaN	4.0	6

	0	1	2	3
1	2.0	3.0	5	NaN

handling

We shouldn't throw away the data, but find the root cause of the problem.

function	method
dropna	drop
fillna	ffill/pad
fillna	backfill/bfill

fill

```
import numpy as np
import pandas as pd

s = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))
print(s.fillna(0))
# print(s.fillna(method='ffill'))
```

```
a    1.0
b    0.0
c    2.0
d    0.0
e    3.0
dtype: float64
```

EXAMPLE

load data

```
import pandas as pd

raw = pd.read_csv('./data/prices.csv')
raw
```

	day	quote	close
0	2015-01-04	NTC	603.0
1	2015-01-01	NTC	604.0
2	2015-01-05	CIT	3020.0
3	2015-01-05	NTC	602.0
4	2015-01-01	CIT	2980.0
...	...	...	...
2377	2019-06-04	NTC	710.0
2378	2019-06-06	NTC	713.0
2379	2019-06-06	SHIVM	678.0
2380	2019-06-06	CIT	2550.0
2381	2019-06-11	NTC	700.0

```
[2382 rows x 3 columns]
```

find missing

```
print(df.isna().sum())  
print(df[df['NTC'].isna()])
```

```
NTC      1365  
CIT      1515  
BNT      1938  
SHIVM    2328  
dtype: int64
```

	NTC	CIT	BNT	SHIVM
2	NaN	3020.0	NaN	NaN
4	NaN	2980.0	NaN	NaN
5	NaN	3035.0	NaN	NaN
6	NaN	3000.0	NaN	NaN
7	NaN	3000.0	NaN	NaN
...	...	...	...	...
2374	NaN	2506.0	NaN	NaN
2375	NaN	NaN	6684.0	NaN
2376	NaN	NaN	NaN	675.0
2379	NaN	NaN	NaN	678.0
2380	NaN	2550.0	NaN	NaN

```
[1365 rows x 4 columns]
```